Avionne Douglas CDA5220 - Programming Assignment 2 Report

All Answers to the Questions are Bolded and begin with ~

Part 1: Warm Up

1.1 Exercise:

Number of Cache Lines	Median Access Latency
1	**0**
10	**0**
100	**0**
1,000	**0.1**
10,000	**0.7**
100,000	**5.5**
1,000,000	**50**
10,000,000	**500**

1.2 Discussion Question:

According to your measurement results, in order to measure differences in time with performance.now(), approximately how many cache accesses need to be performed?

~ According to the measured results, differences in time using performance.now() become measurable and noticeable when a large number of cache line accesses are performed (around 1,000) so the latency exceeds the noise limiting the timer. Accessing fewer cache lines (like around 1–100) produces latencies too small to detect with the JavaScript timer. The latency only starts to become noticeable around a larger range such as 1,000 cache lines. Everything below that (1, 10, 100) shows 0 ms because performance.now() isn't precise enough to capture the very small differences.

Part 2: Side Channel Attacks with JavaScript

2.1 Discussion Question:

Describe your trace collection method and important parameters, such as the number of addresses being accessed *N* and trace length *K*.

~ I implemented a cache-occupancy side-channel attack by repeatedly accessing a large memory buffer and measuring how many times the buffer could be moved within fixed time intervals. Then, the trace was divided into multiple intervals of length P milliseconds, and for each interval I counted the number of complete sweeps of the buffer. This produced a time series representing memory access throughput. Each trace captures how memory access performance changes over time, which is affected by other processes competing for cache resources.

The important parameters I used were:

1. **Buffer size (N): approximately 1,024,000 elements (≈ 1 million). This was chosen because it covers a large portion of the last-level cache.**
2. **Stride: 16 elements. I used this to access different cache lines, ensuring that cache effects are properly captured.**
3. **Interval length (P): 10 ms. This provides a balance between time resolution and measurement noise**
4. **Trace length (K): approximately 500 intervals, covering a total duration of 5 seconds, so that enough temporal information is captured for classification.**

2.2 Exercise:

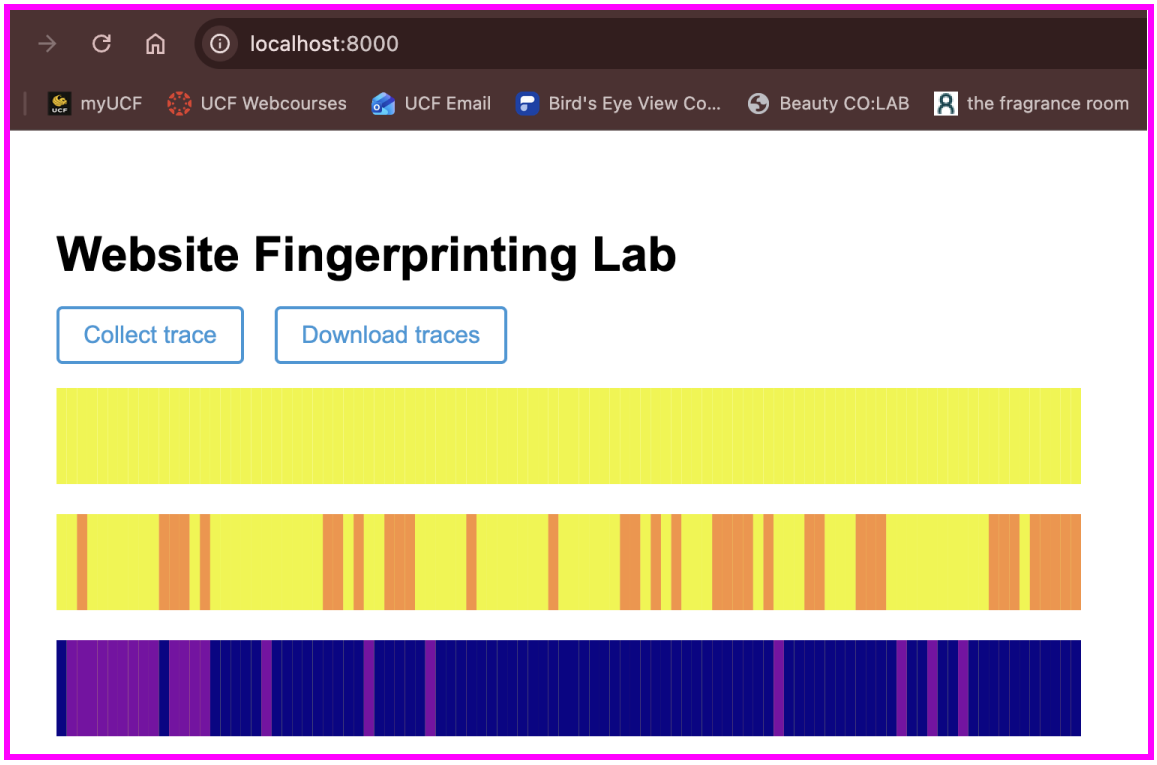
Complete the record() function in worker.js. Collect traces for the three scenarios described above, and take a screenshot of these three traces. Your traces will not exactly match those in the provided example, but they should be visually distinguishable from one another.

~ I implemented the record() function in worker.js to collect cache-occupancy traces by repeatedly accessing a large memory buffer and counting the number of full buffer sweeps completed in fixed time intervals.

I collected traces under three different scenarios as stated in the assignment instructions:

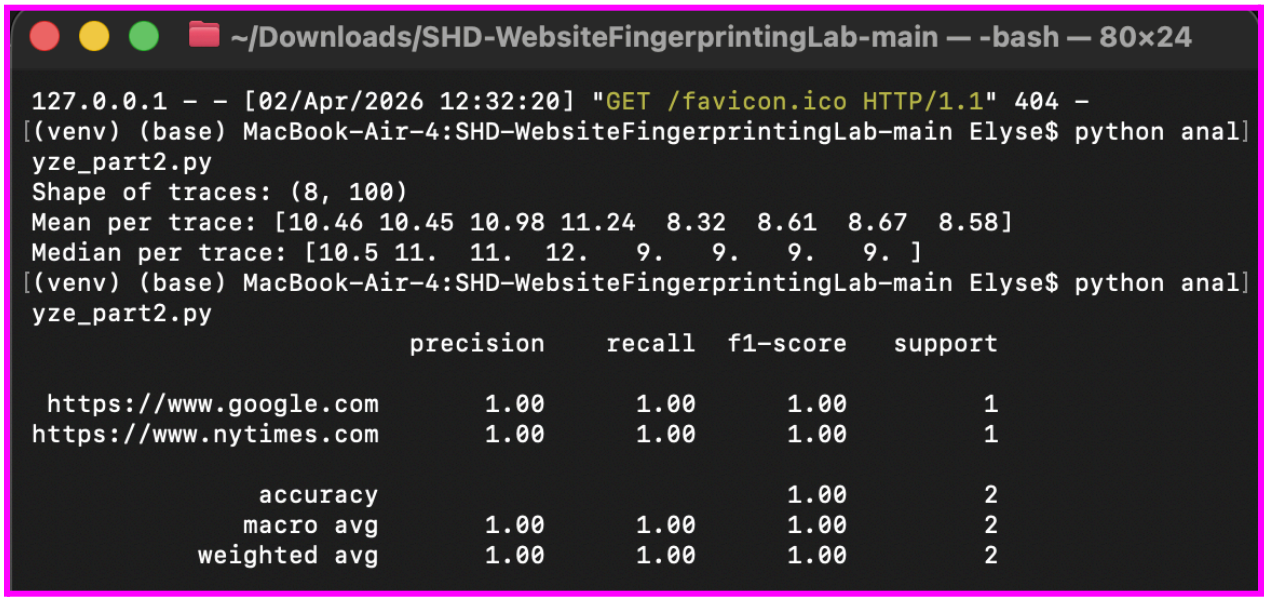
- **Baseline (Do nothing): The trace appears as a uniform yellow bar, indicating consistently high memory access throughput with minimal cache interference.**
- **Random Activity: The trace shows a mix of yellow and orange regions, indicating moderate variation and reduced throughput due to background system activity.**

- **Victim Website: The trace appears mostly dark blue/purple, indicating significantly lower throughput due to heavy cache usage from loading the website.**



2.3 Exercise:

Use the Python code we provided in Part 2.1 to analyze simple statistics (mean, median, etc.) on the traces from google.com and nytimes.com.



2.4 Exercise:

Complete the eval() function in eval.py. In this function you should:

1. Load your traces into memory
2. Split your data into a training and test set
3. Train a classification model on your training set
4. Use your model to predict labels for your test set
5. Print out your model's accuracy using classification_report

```
(venv) (base) MacBook-Air-4:SHD-WebsiteFingerprintingLab-main Elyse$ cd part2
(venv) (base) MacBook-Air-4:part2 Elyse$ python eval.py
                         precision    recall  f1-score   support

    https://www.cnn.com       1.00      0.90      0.95        40
 https://www.google.com       1.00      1.00      1.00        40
https://www.nytimes.com       0.85      1.00      0.92        40
https://www.youtube.com       1.00      0.93      0.96        40

               accuracy                           0.96       160
              macro avg       0.96      0.96      0.96       160
           weighted avg       0.96      0.96      0.96       160

(venv) (base) MacBook-Air-4:part2 Elyse$
```

Part 3: Root Cause Analysis

3.1 Exercise:

Modify record() by removing all memory accesses in your code, and re-collect the traces for the four sites you previously examined. With your new traces, repeat Exercise 2-4 and report the accuracy.

```
index.html                worker.js
(venv) (base) MacBook-Air-4:part3 Elyse$ python eval.py
                         precision    recall  f1-score   support

    https://www.cnn.com       1.00      1.00      1.00        40
 https://www.google.com       0.98      1.00      0.99        40
https://www.nytimes.com       1.00      0.97      0.99        40
https://www.youtube.com       1.00      1.00      1.00        40

               accuracy                           0.99       160
              macro avg       0.99      0.99      0.99       160
           weighted avg       0.99      0.99      0.99       160

(venv) (base) MacBook-Air-4:part3 Elyse$
```

3.2 Discussion Question:

Compare your accuracy numbers between Part 2 and 3. Does the accuracy decrease in Part 3? Do you think that our "cache-occupancy" attack actually exploits a cache side-channel? If not, take a guess as to possible root causes of the modified attack.

~ In Part 2 (full attack with memory accesses) the accuracy is ≈ 0.96, whereas in Part 3 (modified attack without memory accesses) the accuracy is ≈ 0.99. The accuracy in Part 3 increased after removing memory accesses, which was surprising because I expected it to decrease. This suggests that the cache-occupancy attack is not wholly targeting or relying on the cache side-channel. If the attack depended primarily on memory accesses, removing them would likely have caused a clear drop in performance.

One possible explanation is that the attack is picking up patterns of activity that are specific to each website rather than purely cache effects. Another possible cause could be randomness from JavaScript timing and event loop scheduling, which can still create timing differences even without direct memory pressure. Also, browser rendering, network requests, and JavaScript engine behavior might add extra signals that are not actually related to cache usage. So the attack may be picking up regular performance patterns of the websites.

Machine learning side-channel attacks can learn patterns in browser behavior even without direct memory measurements. This shows how powerful ML is at finding hidden signals, but it also shows why it's important to understand what the model is actually learning from.

Part 4: Bypassing Mitigations

4.1 Exercise:

Using your code from Part 2, run the attack with the countermeasure enabled (using the same four websites you previously used). Repeat the evaluation you performed in Exercise 2-4, reporting your accuracy.

```
(venv) (base) MacBook-Air-4:SHD-WebsiteFingerprintingLab-main Elyse$ cd part4
(venv) (base) MacBook-Air-4:part4 Elyse$ python eval.py
                         precision    recall  f1-score   support

     https://www.cnn.com       1.00      0.90      0.95        40
  https://www.google.com       1.00      1.00      1.00        40
 https://www.nytimes.com       0.85      1.00      0.92        40
 https://www.youtube.com       1.00      0.93      0.96        40

                accuracy                           0.96       160
               macro avg       0.96      0.96      0.96       160
            weighted avg       0.96      0.96      0.96       160

(venv) (base) MacBook-Air-4:part4 Elyse$
```

4.2 Discussion Question:

How much did the accuracy decrease compared to Part 2? Is noise injection an effective mitigation? Finally, let's see how much you can do to defeat this countermeasure!

~ In Part 2 the accuracy was about 0.99, while in Part 4 (with noise) it dropped to about 0.96. So the accuracy decreased by around 3%, which means the countermeasure had some effect. The noise injection does reduce the classifier's performance a little, but the attack still stays very high in accuracy (above 80%), so it is not a very strong defense on its own. The classifier can still pick up patterns in the traces even with the added noise, so it can still do well despite the countermeasure.

4.3 Discussion Question:

Propose a change to the attacker - you may consider changing the attacker's algorithm or variable passed to the automation script (such as the trace length). Explain why your method should be effective in theory.

~ I increased the trace length from 5000 ms to 7000 ms so each trace has more data. Longer traces give more samples per website, which helps reduce the effect of the random noise added by the countermeasure. With more data, the noise gets "smoothed out" more, so the overall pattern becomes easier to see. Another idea is that increasing the number of sweeps or the buffer size in the worker.js file could also make the signal stronger compared to the noise.

The countermeasure adds random noise to make classification harder. By collecting longer traces, we get more consistent information per website, which helps cancel out the noise effect. This makes it easier for the classifier to tell the difference between websites again.

4.4 Exercise:

Evaluate your proposed change by repeating the steps from Exercise 4-1. Did your accuracy increase?

```
(venv) (base) MacBook-Air-4:SHD-WebsiteFingerprintingLab-main Elyse$ cd part4
(venv) (base) MacBook-Air-4:part4 Elyse$ python eval.py
                        precision    recall  f1-score   support

    https://www.cnn.com       0.90      0.95      0.93        40
 https://www.google.com       1.00      0.88      0.93        40
https://www.nytimes.com       0.95      1.00      0.98        40
https://www.youtube.com       0.98      1.00      0.99        40

              accuracy                           0.96       160
             macro avg       0.96      0.96      0.96       160
          weighted avg       0.96      0.96      0.96       160

(venv) (base) MacBook-Air-4:part4 Elyse$
```

~ After increasing the trace length to 7000 ms, I re-ran the attack under the same countermeasure settings. The accuracy stayed about the same at around 0.96 compared to Part 4.1. This shows that just increasing the trace length doesn't really improve the results under the noise-based countermeasure. The added noise is still affecting the signal, and the classifier is already picking up most of the useful pattern in the traces. This suggests the countermeasure only gives limited protection, since the attack still performs well even after changing how the data is collected.